OpenClaw Navigation Transport

OpenClaw Navigation Transport

1. Course Content
2. Preparation
3. Case Demonstration
4. Parameter Debugging and Source Code Analysis
   - 4.1 Startup Parameter Description
   - 4.2 Key Parameter Description

1. Course Content

- This lesson is a comprehensive lesson, combining robotic arm visual grasping and intelligent navigation functions for complex long-process tasks. Please first complete the basic lessons in [OpenClaw Arm Tracking And Grasping], [OpenClaw SLAM Mapping And Navigation], and [Road Network Planning and Navigation].

2. Preparation

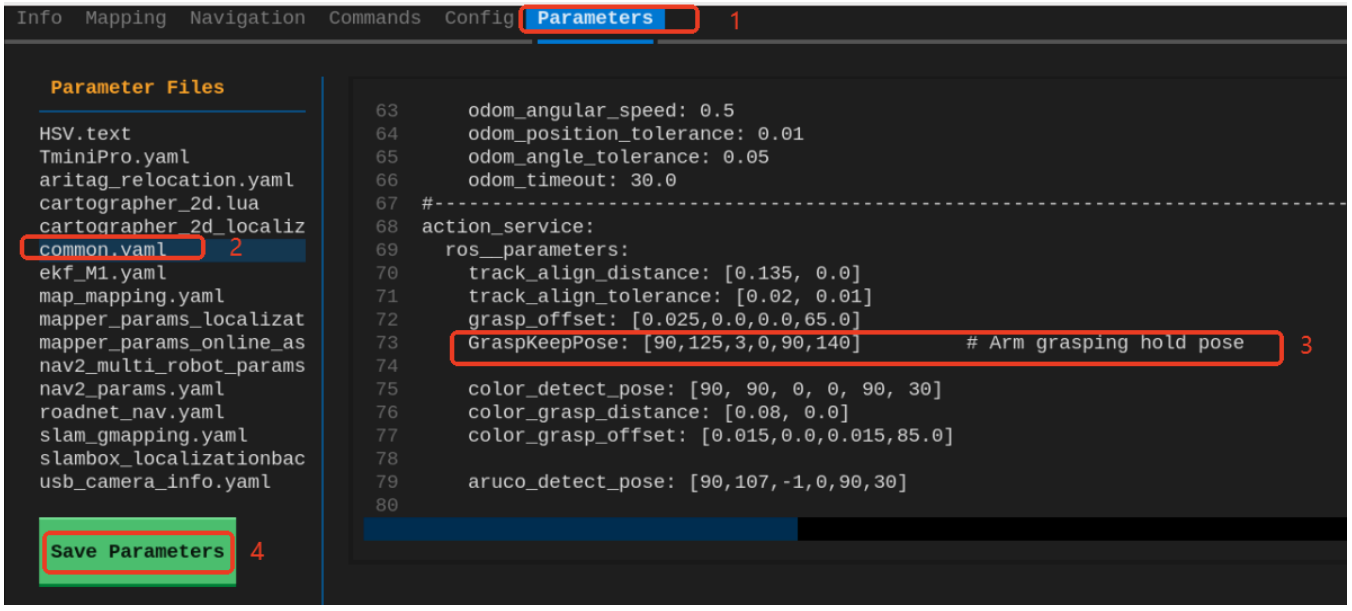
- Complete the preceding tutorials and master the map building, road network annotation, and map mapping markers functions in advance.
- Start nodes such as odometry, tf, robotic arm assist, and camera. If you need to enable the visual relocalization function, you need to add the startup parameter relocalization. The demonstration here uses starting relocalization as an example; if relocalization is not needed, you can remove the relocalization parameter.

```
ros2 launch yahboomcar_bringup mcp_service.launch.py use_camera:=true use_nav:=true
relocalization:=true
```

- Demonstration props: 4 * 4 * 4cm foam color blocks



- If the block size you use is different, you can test the gripper's tightness when grasping in advance and modify the corresponding GraspKeepPose. The last element of the array is the hold angle of the gripper when grasping.



3. Case Demonstration

> **Tip**
>
> - You can freely edit the command, the color blocks used, the semantic locations, etc. according to the actual situation. The following is a reference case.
> - You can specify a position in your own built map and record the semantic map point into the map mapping file.
> - The depth camera version can grasp any target through tracking, while the USB monocular camera can only grasp color wooden blocks.

- Transport a purple block from the raw material warehouse to the blue shelf area of the machining center.

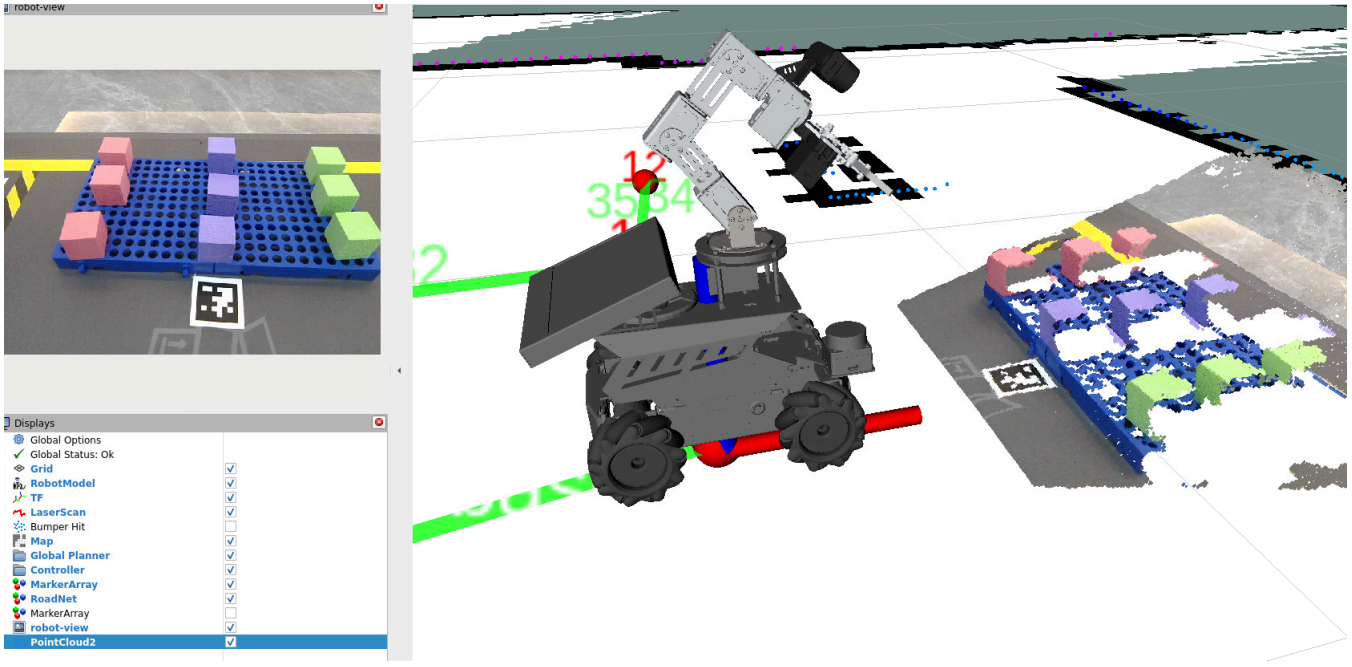
- Mark the map mappings of "raw material warehouse" and "machining center" in advance (for the marking method, refer to the preceding lesson [OpenClaw SLAM Mapping And Navigation]).

```
E:
  name: Raw Material Warehouse
  position:
    x: 0.155
    y: -2.047
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.732
    w: 0.681
F:
  name: Machining Center
  position:
    x: 2.422
    y: -2.665
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.707
    w: 0.707
```

- Then give OpenClaw the specified task; just choose any one of the interaction methods.

```
connected | idle
agent main | session main (openclaw-tui) | bailian/qwen3.6-plus | tokens ?/1.0m

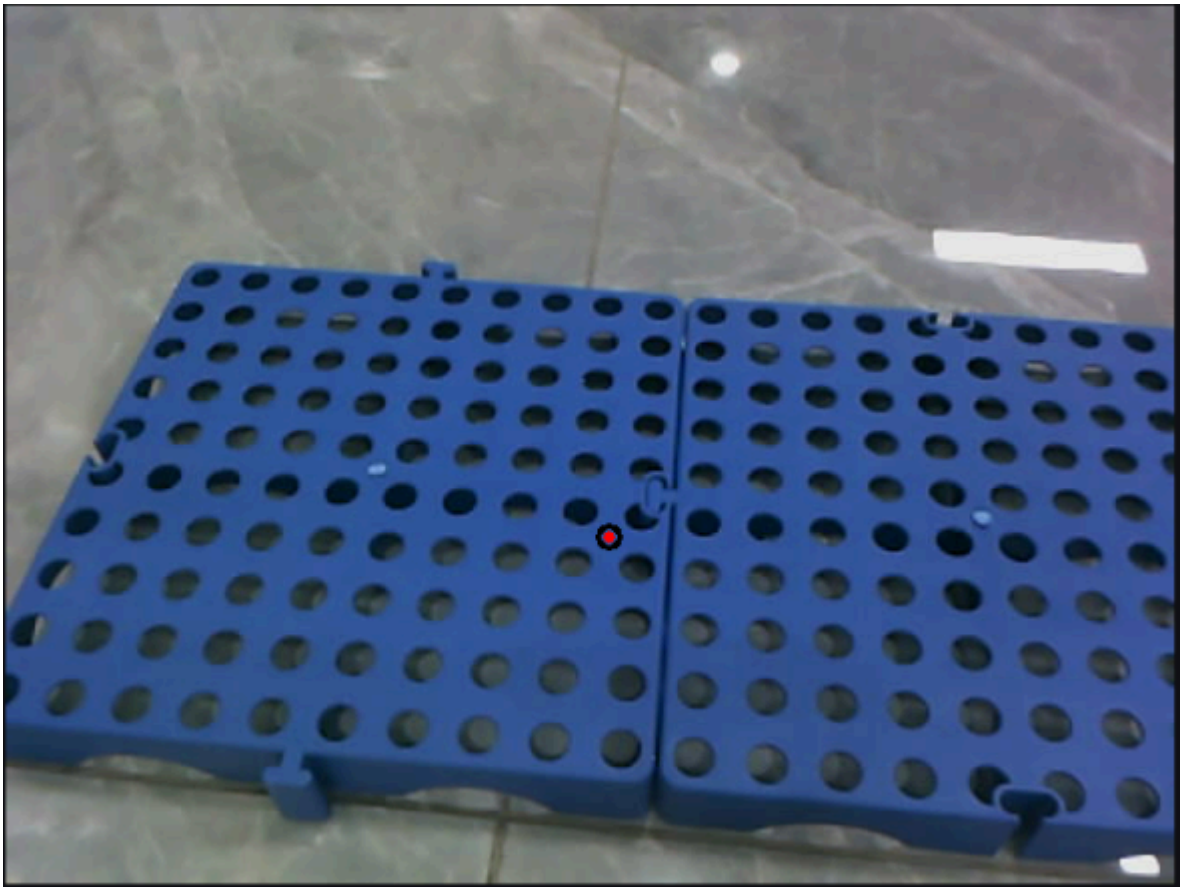
Move a purple cube from raw material warehouse to the machining center's blue shelf area
```



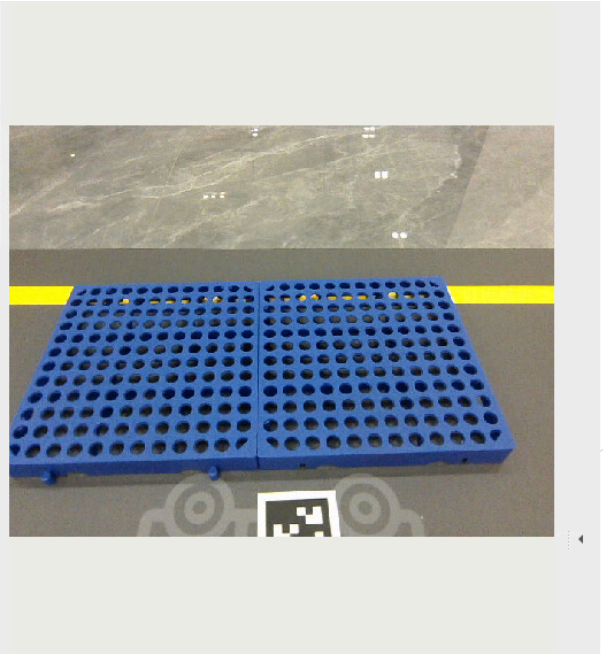
- OpenClaw will call the tool to first grasp the color block and then navigate to the location for placement.

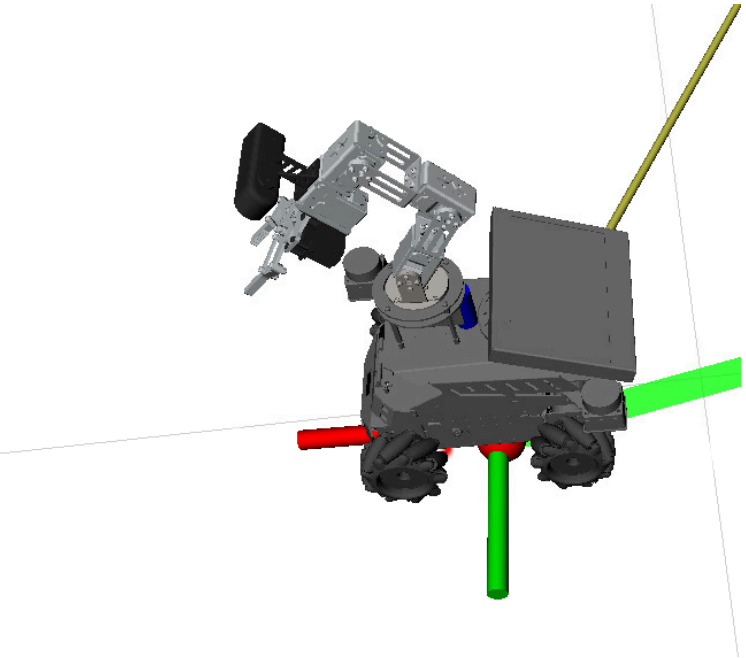
> **Note**
>
> Different camera versions will have different placement effects. The depth camera can obtain the spatial coordinates of the placement point for fixed-point placement, while the USB monocular camera performs placement through a fixed action group.

- The depth camera first obtains the spatial coordinates of the placement point through the vision model and then performs placement.



- The USB camera performs the placement action through the fixed action putdown_joints (you can modify and customize the action through putdown_joints in the common.yaml parameter file).





4. Parameter Debugging and Source Code Analysis

- This lesson is a comprehensive lesson. For the debugging parameters and function source code, see [OpenClaw Arm Tracking And Grasping] and [OpenClaw SLAM Mapping And Navigation].

4.1 Startup Parameter Description

- `use_camera:=true`: Enable the camera node, either the depth camera or the USB monocular camera, for visual grasping and detection.
- `use_nav:=true`: Enable navigation-related nodes (odometry, TF transformation, costmap, etc.), supporting SLAM navigation functions.
- `relocalization:=true`: Enable the ArUco marker visual relocalization function. After navigation arrival, the global localization is corrected through the marker, improving placement precision.

4.2 Key Parameter Description

The following parameters are located in `common.yaml` and are directly related to the navigation transport task:

- `GraspKeepPose`: The grasping hold pose of the robotic arm, in the format `[J1, J2, J3, J4, J5, J6]`, unit: degrees. J6 is the hold angle of the gripper when grasping (0-180); the larger the value, the tighter the gripper grips. If the block size used is different, J6 needs to be adjusted to ensure the gripper can firmly grip the object.
- `putdown_joints`: The fixed placement action group joint angles of the USB monocular camera, in the format `[J1, J2, J3, J4, J5]`, unit: degrees. The USB camera version completes placement through this action group; the depth camera version does not need this parameter.
- `auto_back_dist`: The automatic retreat distance, unit: meters. After grasping/placement is complete, the chassis automatically retreats this distance to avoid collisions and entering the costmap restricted area. It is recommended to appropriately increase it in the navigation transport scenario.
- `place_distance`: The placement position reference and working range detection, in the format `[x`

`reference, y reference, maximum mechanical distance on x axis, maximum mechanical distance on y axis]`, unit: meters. For depth camera fixed-point placement, it is used to determine whether the target point is within the reachable range of the robotic arm.

- `place_offset`: The placement position offset, in the format `[x offset, y offset, z offset, placement pitch angle]`, used to fine-tune the coordinates of depth camera fixed-point placement.
- `grasp_distance`: The grasping alignment distance, in the format `[desired distance in x direction, desired distance in y direction, yaw angle]`, the desired distance maintained when the chassis aligns with the target.
- `grasp_offset`: The grasp position offset, in the format `[x offset, y offset, z offset, grasp angle]`, used to fine-tune the grasp coordinates.
- `odom_linear_speed` / `odom_angular_speed`: The linear velocity (m/s) and angular velocity (rad/s) of the chassis during navigation transport.
- `odom_position_tolerance` / `odom_angle_tolerance`: The position tolerance (meters) and angle tolerance (radians) of the chassis odometry movement.
- `tf_tolerance`: The TF transformation tolerance, unit: seconds.